



How the CTP Portal Actually Works

Computer Training Program · Zonal Coordination

A plain-language tour of the national centre directory — how the software is put together, where your information is kept safe, how it reached the internet, and why running it costs the organisation nothing at all.

- 01 How is the code written?
- 02 Where is the data saved?
- 03 How was it deployed?
- 04 Why is it free?

Live now at

ctp-zonal-coordination.onrender.com

₹0

PER MONTH

1

How the code is written

Three simple layers, each with one job

Think of the portal as a **shop**. There is the **counter** customers see, the **staff** behind it who know the rules, and the **stock room** at the back. Every piece of the software falls into one of those three layers — and each only ever talks to the layer next door.



The Counter

BROWSER

What a visitor actually sees and clicks: the India map, the centre directory, forms, photo galleries and the news band. It runs **inside the visitor's own browser** and holds no secrets.

index.html

styles.css

core.js

map.js

pages/

▼ asks for data



The Staff

SERVER

The rule-keeper. It checks who you are, decides what you may see or change, validates every form, hides private phone numbers from the public, and files everything away correctly.

Nothing reaches storage without passing through here.

server.js

auth.js

models.js

routes/

geo.js

▼ saves & fetches



The Stock Room

STORAGE

Where centres, sessions, certificates and photographs actually live. The rest of the app never cares *which* stock room is in use — that is the trick that makes free hosting possible (see page 2).

store.js

db.js

blobs.js

What that looks like on disk

CTP-Zonal-Coordination/

```
├─ server.js           the front door
├─ server/            the staff: rules, logins, saving
│  └─ routes/         9 sections of the app
├─ public/            the counter: everything visitors see
│  └─ js/pages/       8 screens
├─ data/ uploads/     the stock room, when run at home
├─ render.yaml        hosting instructions
└─ smoke-test.ps1     153 automatic checks
```

36

SOURCE FILES

7,681

LINES OF CODE

5

OUTSIDE PARTS

153

AUTO CHECKS

0

BUILD STEPS



Deliberately boring — and that is the point

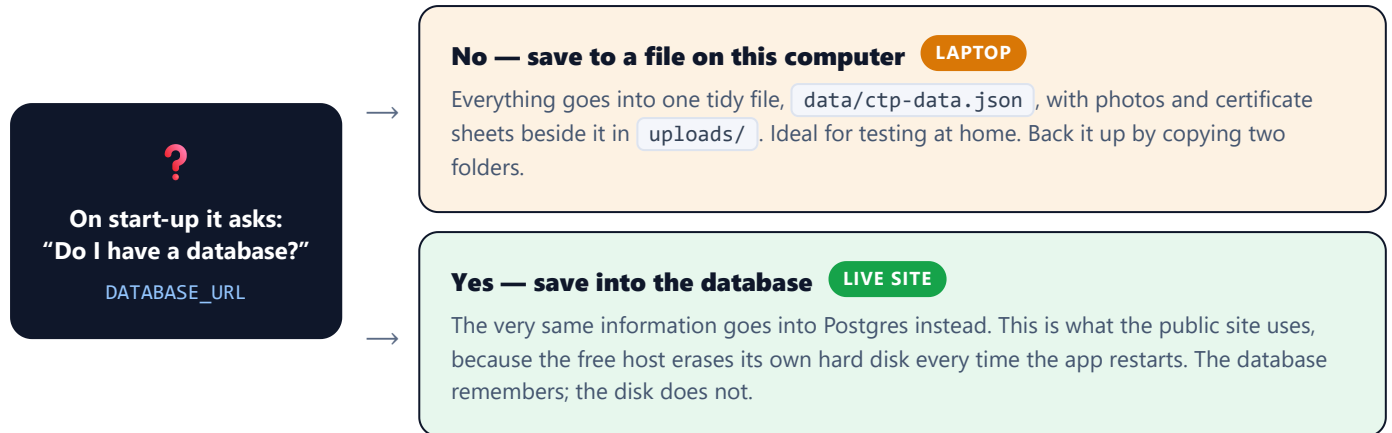
The portal is plain JavaScript with only five outside parts. There is no complicated build machinery to break, no framework to go out of date, and no specialist needed to keep it alive. Any developer can open the folder and read it top to bottom. A volunteer-run charity should never depend on software that only one person understands.

2

Where the data is saved

One app, two stock rooms — chosen automatically

The portal can keep its information in **two different places**, and it decides which one to use the moment it starts. Nobody chooses in the code — you simply tell it whether a database is available.



What is kept, and in which drawer

DRAWER	WHAT GOES IN IT	NAME ONLINE
The record book	Centres, addresses, zones, coordination teams, volunteers, sessions, certificate reflections, enrollments, news and logins	<code>ctp_state</code>
The filing cabinet	Uploaded files — centre photographs and certificate spreadsheets, stored as the actual bytes	<code>ctp_blobs</code>
The safety net	Automatic snapshots taken <i>before</i> anything destructive happens, so a mistake can always be undone	<code>ctp_backups</code>

How your data is protected



Passwords are never stored

Only a scrambled fingerprint is kept. Even someone holding the whole database cannot read anyone's password — not even yours.



Private stays private

Phone numbers, emails and volunteer rosters are removed before anything is sent to the public — not merely hidden on screen.



Nothing half-written

Saves are all-or-nothing and happen one at a time, so a power cut can never leave the records in a broken state.



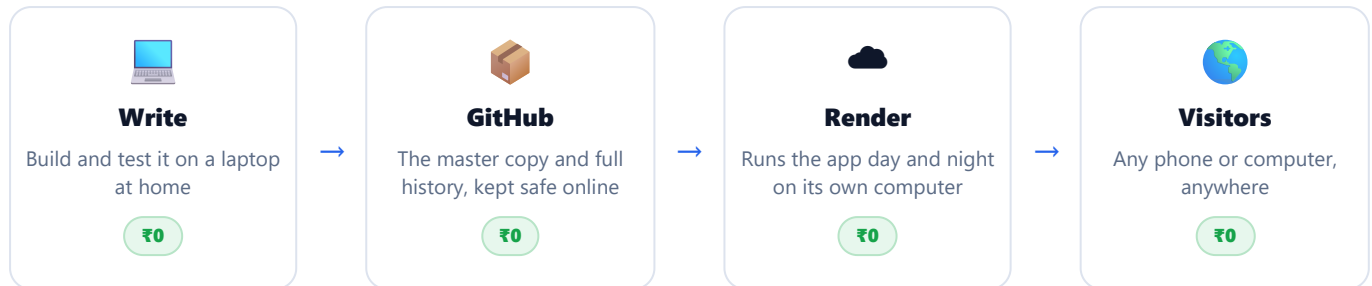
Taking a backup takes one click

Admin console → **Backup** downloads the entire portal as a single file you can keep on a pen drive or in email. Password fingerprints are deliberately left out of that file, so it is safe to store and share internally.

How the website was deployed

From a laptop to the whole of India in four hops

“Deploying” means moving the software off the computer it was written on, onto a machine that stays switched on for everybody else. Here is the whole journey.



Running alongside, **Neon** holds the database — also free — so nothing is lost when Render restarts.

What actually happened, step by step

- 1 The code was published to GitHub** as a public repository under the MIT licence, so the work is permanently preserved and anyone may reuse it.
- 2 A free Neon database was created** in Singapore — the nearest region to India — and its connection details were handed to the host as a secret.
- 3 Render was pointed at the repository.** A small instructions file in the project, `render.yaml`, already told it everything (shown below).
- 4 The administrator account was created** on first boot from settings held in Render's dashboard, never written into the code.
- 5 It went live** — and now every change pushed to GitHub rebuilds and republishes the site automatically, with no manual uploading ever again.



Updating takes one command

A change is saved to GitHub, Render notices within seconds, rebuilds and swaps the new version in — two to three minutes, no downtime. Exactly how the centre-photo feature reached the live site.



Nothing ships untested

Before any release, **153 automated checks** run through the portal — logins, centres, photo uploads, and confirming people cannot see data that is not theirs. Run against *both* storage modes. All 153 passed.

The entire hosting instruction, in eight settings

```
# render.yaml — Render reads this and needs nothing else
type: web      runtime: node    plan: free
region: singapore ← nearest to India
buildCommand: npm ci ← fetch the five outside parts
startCommand: npm start ← switch the portal on
healthCheckPath: /api/health ← keep checking it is alive
envVars: DATABASE_URL, CTP_ADMIN_EMAIL, CTP_ADMIN_PASSWORD
      ↑ only the names of the secrets live here. The values are typed into
      Render's dashboard — which is why the project can safely be public.
```

Why it costs ₹0

Several generous free tiers, stitched together

The portal is not free because corners were cut. It is free because every piece was chosen from a service that gives away a small amount permanently — and a charity directory fits comfortably inside “small”.

PIECE	PROVIDED BY	COST	WHY IT IS FREE FOR US
The app itself	Render free plan	₹0	Free for small projects, and no credit card is ever requested.
The database	Neon Postgres	₹0	0.5 GB free — far more than this directory will ever need.
Code storage	GitHub public repo	₹0	Public open-source projects are hosted free, with unlimited history.
The India map	OpenStreetMap	₹0	Community-made map data. No Google Maps billing account needed.
Security certificate	Render, automatic	₹0	The padlock and <code>https://</code> are issued and renewed for us.
Software licences	Open source	₹0	All five outside parts are free to use, including commercially.
Total	—	₹0 / month	No card on file. No trial that expires.

● The one honest trade-off



The site naps when nobody is using it

After **15 minutes** with no visitors, the free plan puts the app to sleep. The next person to arrive waits roughly **30–60 seconds** while it wakes up; everyone after them gets an instant site. **Nothing is lost** during a nap — the data sits in the database, not on the sleeping machine.



Fix it for free

A free UptimeRobot monitor visiting the site every 5 minutes keeps it awake through the working day. Still ₹0.



Or pay to remove it

About **\$7 a month** on Render's Starter plan removes the sleep entirely and allows a custom web address.



Nothing is locked in

The same code runs on Azure, on a college server, or on a laptop. Changing host needs no code changes at all.



Why a charity directory fits inside “free” so comfortably

Free tiers are sized for hobby projects, and this portal is genuinely small. Even the whole national directory with photographs occupies a small fraction of the **0.5 GB** the database gives away.

What this would normally have cost

THE USUAL WAY	TYPICAL COST	OUR COST
An agency builds a custom web portal	a large one-off fee	₹0
Web hosting plan	monthly subscription	₹0
Managed database	monthly subscription	₹0
Google Maps usage	billed per 1,000 map loads	₹0
Annual maintenance contract	yearly retainer	₹0

“What if one of these free services stops?”

IF THIS CHANGED...	WHAT IT WOULD TAKE TO RECOVER
Render stops being free	Move to another host. The project already carries instructions for Azure, and the same code runs unchanged on any computer with Node installed.
Neon stops being free	Point <code>DATABASE_URL</code> at any other Postgres database and restart. Postgres is a 30-year-old open standard, not a proprietary product.
GitHub changes	A full copy of the code and its entire history already sits on the laptop it was written on, and on every machine that has ever downloaded it.
Everything goes at once	The one-click backup file plus the public code rebuilds the portal from scratch on a laptop in minutes.

The three accounts that keep it alive

ACCOUNT	WHAT IT HOLDS	IF ACCESS TO IT IS EVER LOST
GitHub	The code and its full history	Nothing breaks — the live site keeps running.
Render	The running website and its three secrets	Make a new free service, point it at the code, retype the secrets.
Neon	The actual data	The one that matters. A recent backup file restores it anywhere.



The organisation owns everything

The code is yours under the MIT licence, the data exports with one click, and no part of the portal depends on a paid product, a trial, or a vendor's private format. Free here means *unencumbered*, not *borrowed*.



The whole thing on one page

Everything above, in sixty seconds



How it is built

Three layers — what you see, the rules behind it, and storage. Plain JavaScript, five outside parts, no build machinery, readable by any developer.



Where data lives

A free Neon Postgres database in Singapore holds records, photos and automatic backups. A simple file is used instead when testing at home.



How it got online

Laptop → GitHub → Render → the public. Every saved change republishes the site automatically in two to three minutes.



Why it is free

Permanent free tiers, stitched together. No credit card, no expiring trial. The only trade-off is a short wake-up after a quiet spell.

Looking after it

IF YOU WANT TO...	DO THIS
Take a backup	Admin console → Backup , then save the file somewhere safe.
Add a centre	The coordinator registers on the site; an administrator approves them.
Publish news	Admin → Announcements. It appears in the band on the home page at once.
See it filled with examples	Admin → Load demo data , which installs a sample national directory.
Keep it awake	Point a free UptimeRobot monitor at <code>/api/health</code> .
Use your own web address	Render → Settings → Custom domains, then add the record it shows you.



Built so it outlives whoever set it up

The code is public and MIT-licensed, the data exports with one click, the host can be swapped without touching a line, and 153 automated checks document exactly how the portal is meant to behave. Nothing here depends on one person staying available — which is the kindest thing software can do for a volunteer organisation.

ctp-zonal-coordination.onrender.com

Source code: github.com/Sahil797/ctp-zonal-coordination · MIT licence